Informe Diseño de arquitectura

– ENTREGA 1 –

Asignatura
Fullstack III

Sección
DSY1106-002D

Profesor
Daniel Williams Concha

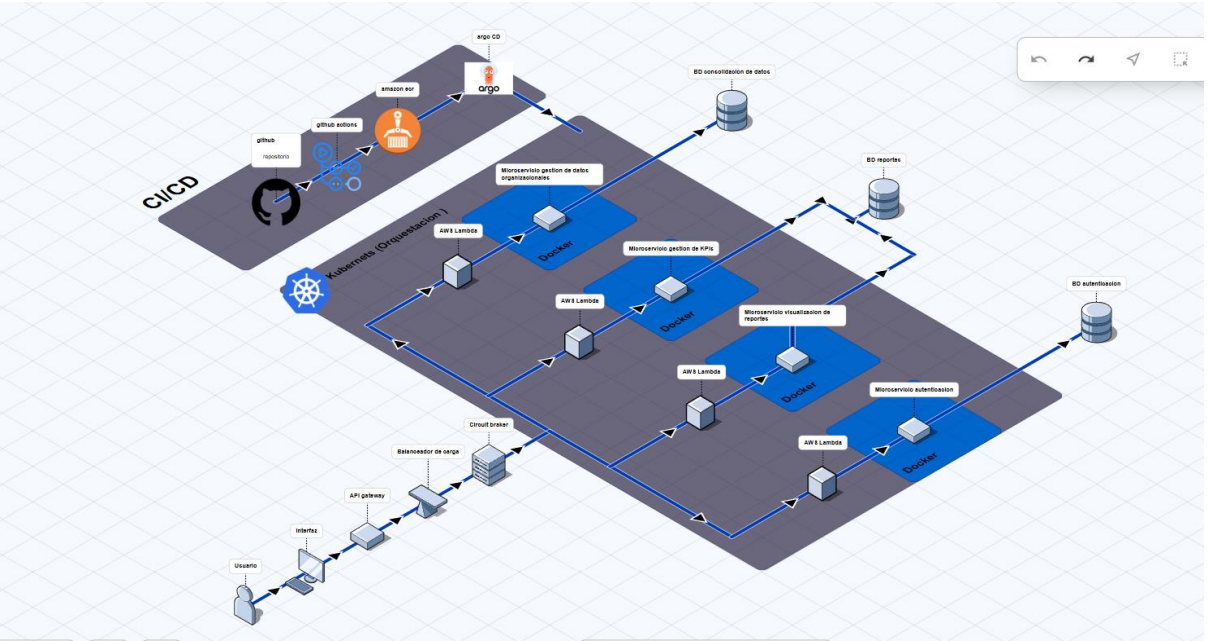
Integrantes
Tomás Cortés
Emmanuel Opazo

Concepción, 08 de abril de 2026

1.-Contexto

El **Grupo Cordillera** sufre fragmentación de datos operativos debido al uso de 5 sistemas legados aislados (Sistemas de punto de venta, Plataformas de comercio electrónico, Sistemas de gestión de inventarios, Herramientas de gestión financiera y Sistema de atención al cliente). Se requiere implementar una **Plataforma de Monitoreo Inteligente** que consolide esta información en tiempo real para visualizar KPI's y reportes, permitiendo a la alta gerencia tomar decisiones estratégicas ágiles.

2.-Diagrama de arquitectura



El diagrama muestra una arquitectura de microservicios en Kubernetes. El flujo inicia en sistemas externos y Frontend, atraviesa una capa de seguridad (API Gateway/BFF) y llega a los servicios de dominio, los cuales cuentan con persistencia propia y observabilidad automatizada.

3.-justificación de tecnología usada

Tecnología / Herramienta	Función en la Arquitectura	Justificación de su elección

Java + Spring Boot (Maven)	Framework base del Backend y gestor de dependencias.	Posee un ecosistema maduro y nativo para construir microservicios. Maven permite usar arquetipos para estandarizar el código de todo el equipo.
React (NPM)	Tecnología del Frontend Modular.	Permite crear interfaces dinámicas y carga diferida, ideal para renderizar dashboards de KPIs pesados sin lentitud.
PostgreSQL	Motores de base de datos relacional y en memoria.	PostgreSQL asegura la consistencia de los datos financieros/ventas consolidados. Caché de Aplicación evita recalcular reportes, entregando KPIs a hipervelocidad.
Docker y Kubernetes (K8s)	Contenerización y orquestación de la infraestructura.	Docker asegura que el código funcione igual en cualquier entorno. K8s automatiza despliegues y realiza auto-escalado si hay picos masivos de uso.
Git , Git Actions, Amazon ecr y Argo CD	Herramientas del pipeline CI/CD.	El flujo comienza en **GitHub**, donde el código fuente se almacena y versiona; al realizar un cambio, se activa **GitHub Actions**, que compila el proyecto, ejecuta pruebas y construye una imagen **Docker**. Esta imagen se almacena en **Amazon ECR**, funcionando como un registro seguro en la nube de AWS, para que finalmente **Argo CD** detecte la nueva versión y sincronice automáticamente el clúster de **Kubernetes**, desplegando los cambios de forma controlada y eficiente

4.-Tecnología de orquestación

Docker y Kubernetes (K8s): Todo el sistema se empaqueta en Docker. K8s orquesta estos contenedores, automatizando el despliegue y el **auto-escalado**. Ante picos de demanda (ej. Cyber Monday), levanta réplicas de los servicios afectados para mantener la disponibilidad sin intervención manual.

5.-Justificación microservicio vs monolítico

Se optó por microservicios por dos ventajas críticas:

1. **Escalabilidad independiente:** Permite inyectar más recursos de CPU solo al "Motor de KPIs", sin costear la escalabilidad de todo un monolito.
2. **Despliegues sin inactividad:** Aísla fallos. Si el módulo de Reportes se actualiza o falla, la recolección de datos no se detiene.

6.-Métodos HTTP

Método HTTP	Nombre del Proceso	Qué hace en el sistema
GET	Obtener Dashboard	Lee la estructura visual desde el BFF para el usuario.
GET	Listar KPIs	Trae indicadores pre calculados desde la Caché de Aplicación.
GET	Estado de Consolidación	Consulta si el último lote de datos se procesó.

GET	Validar Sesión	Verifica vigencia del Token JWT del usuario.
POST	Iniciar Sesión	Envía credenciales seguras y devuelve un JWT.
POST	Ingestar Ventas	Envía nuevos lotes de datos desde el Punto de Venta.
POST	Crear Plantilla Reporte	Guarda configuración de un nuevo reporte personalizado.
PUT	Actualizar Fórmula KPI	Modifica parámetros matemáticos de un indicador existente.
PUT	Actualizar Rol Usuario	Cambia los permisos de un usuario.
DELETE	Eliminar Reporte	Desactiva o borra lógicamente un reporte personalizado que la gerencia ya no necesita visualizar.

7.-Microservicios identificados y sus Funciones

Microservicio identificado	Responsabilidad en la plataforma
Microservicio de Gestión de Datos Organizacionales	Recibe datos en bruto de los 5 sistemas legados, los limpia y almacena en PostgreSQL.
Microservicio Motor de KPIs	Toma datos limpios, aplica fórmulas de negocio y guarda los resultados en Caché
Microservicio de Visualización de Reportes	Da formato visual a los KPIs calculados para que el frontend los grafique fácilmente.
Microservicio de Autenticación y Seguridad	Válida identidades (JWT) y aplica Control de Acceso por Roles.

8.-Justificación y Patrones conectados entre servicios

Patrón de Diseño	Dónde se aplica en la Arquitectura	Justificación de su uso en el proyecto
Repository Pattern	Capa de persistencia.	Desacopla la lógica de negocio del acceso a datos (JPA). Si el motor de BD cambia, el núcleo del servicio queda intacto
Factory Method	Motor KPIs y Reportes	Delega la creación de indicadores a clases especializadas. Permite

		sumar nuevos tipos de KPIs a futuro con flexibilidad.
Circuit Breaker	Conexiones internas.	Evita fallos en cascada. Si un servicio cae, "abre el circuito" impidiendo que el BFF se bloquee esperando respuestas.
Backend For Frontend (BFF)	Entre el API Gateway y los Microservicios	Orquesta peticiones; junta información de varios microservicios y devuelve una sola respuesta adaptada al frontend.
API Gateway	En la entrada principal del clúster de Kubernetes, antes del BFF.	Centraliza ruteo, fuerza HTTPS y mitigar ataques.

10.-Evaluación Ética

- **Seguridad:** Uso de tokens JWT de vida corta, cifrado en reposo para bases de datos y protección contra ataques de denegación de servicio (DDoS) mediante el Gateway.
- **Privacidad:** Implementación de roles. Garantiza que los ejecutivos solo vean la información autorizada para su rol jerárquico, cumpliendo normativas de protección de datos.
- **Cifrado en Reposo:** Los datos almacenados en **PostgreSQL** no están en texto plano; se utiliza el estándar de cifrado avanzado para que, aunque alguien robara físicamente los archivos de la base de datos, no pudiera leer la información de ventas o clientes.

- **Sostenibilidad:** El **auto-escalado** de Kubernetes reduce la cantidad de servidores activos en horarios nocturnos, ahorrando energía. El uso de **Caché de Aplicación** evita recalcular reportes, disminuyendo el consumo de CPU y la huella de carbono.

11.-Conclusión

La arquitectura propuesta para la Plataforma de Monitoreo Inteligente resuelve eficazmente la fragmentación operativa del Grupo Cordillera al integrar sus cinco sistemas legados en un ecosistema de microservicios robusto, escalable y seguro. El uso de tecnologías de alta disponibilidad como Spring Boot, React y Kubernetes, junto a patrones estratégicos como API Gateway y Circuit Breaker, garantiza la entrega de KPIs en tiempo real a la gerencia, erradicando los procesos manuales y asegurando la continuidad operativa ante fallos. Finalmente, la solución destaca por su estricto cumplimiento de estándares éticos y de sostenibilidad —mediante control de accesos, cifrado de datos y prácticas de ahorro energético—, posicionándose como una infraestructura sólida, privada y eficiente preparada para las futuras fases de desarrollo de la compañía